

Pesquisa Profunda — Studify: Funcionalidades Inovadoras, IA, Visual e Arquitetura Modular

Data: 27/08/2026 | Branch: Teste/correções @ 283ece6 (Telas `profile/progresso` criadas revitalizadas;bugs corrigidos) Stack: Expo ~53.0.22, React 19.0.0, React Native 0.79.6, expo-sqlite 15.2.14, expo-crypto, AsyncStorage, React Navigation/stack, Groq `llama-3.3-70b-versatile` Objetivo: Onde integrar IA de verdade, que recursos visuais usar, e como modularizar para escalar sem virar spaghetti. Sem viagem, sem enchimento.

1. Diagnóstico Real do Studify Hoje

O que já existe e funciona

Camada	Arquivos	O que faz
Auth	<code>src/services/authDb.js</code>	SQLite <code>users(id, email, senha_hash, avatar)</code> , hash SHA256 + salt 16 bytes (base64), sessão em <code>AsyncStorage</code> (<code>studify_session</code>), <code>registerUser</code> com <code>INSERT OR IGNORE</code> anti-race, <code>verifyPassword</code> com legacy fallback

Matérias/Sessões	<code>src/services/subjectsDb.js</code> (~900 linhas)	<code>subjects(id, nome, topicos</code> <code>JSON, fixada, created_at,</code> <code>accessed_at, user_id) +</code> <code>sessions(subject_id,</code> <code>started_at,</code> <code>duration_minutes, user_id)</code> <code>+</code> <code>chat_conversations/messages</code> <code>+</code> <code>user_settings/badge_definitions/user</code> Funções: <code>carregarMaterias,</code> <code>criarMateria,</code> <code>atualizarMateria</code> (whitelist), <code>registrarSessao,</code> <code>carregarHistorico,</code> <code>getProfileStats,</code> <code>checkAndAwardBadges,</code> <code>getWeeklyGoalProgress</code>
Home	<code>src/screens/HomeScreen.js,</code> <code>components/home/*</code>	Busca <code>filterRevisados</code> , histórico slice 10, fixar (<code>toggleFixadaDb</code>), criar/editar matéria (max 10 tópicos), <code>ProgressCards,</code> <code>MateriaSections,</code> <code>CardMateria, BottomNav</code>
Detail	<code>src/screens/DetailScreen.js</code>	Timer real <code>setInterval</code> 1s com pause/resume, <code>clearTimer</code> com cleanup, registra <code>Math.floor(elapsed/60)</code> só se <code>>=1min</code> , toggle tópico <code>estudado</code> , progresso círculo + lista sessões
Chat IA	<code>src/services/aiService.js,</code> <code>src/screens/ChatScreen.js</code>	Groq <code>llama-3.3-70b-versatile</code> 512 tokens temp 0.7, system prompt educacional PT-BR max 4 parágrafos, histórico SQLite <code>chat_conversations</code> (limite 20 com <code>limparConversasAntigas</code>), título auto 40 chars, <code>reqTokenRef</code> anti-race

Profile/Progress

`src/screens/ProfileScreen.js`,
`ProgressScreen.js`

`getProfileStats()` (totalHoras, topicosConcluidos, materiaTop, melhorDia, mediaSessao, streak),
`getWeeklyGoalProgress()` (goal vs current %), streak calcula
`datasUnicas` ordenadas vs `hoje`,
seed 8 badges (first_session, one_hour, streak_7/30, collector, topic_master, marathoner), gráfico semanal 7 dias barras

Gargalos que bloqueiam inovação

1. **IA sem contexto:** `aiService.enviarMensagem()` recebe só `role/text` genérico. Não sabe as matérias/tópicos/horas/streak do usuário. É um ChatGPT trancado em prompt educacional, nunca vai dizer "você travou em Termodinâmica 60%".
2. **Timer desconectado de aprendizado:** `sessions` salva só `duration_minutes`. Não gera revisão, não alimenta algoritmo. Tempo vira estatística morta.
3. **Tópicos são strings burras:** `topicos: [{nome, estudado: bool}]` — sem dificuldade, sem prioridade, sem `due_date`. Impossível fazer spaced repetition real.
4. **Arquitetura flat:** `src/screens + src/components/home + src/services + src/styles`. `subjectsDb.js` faz tudo (subjects + sessions + chat + badges). `authDb` e `subjectsDb` abrem o MESMO `studify.db` com `getDb()` duplicado (risco de race). App com 9 screens já no limite do tangled.

2. Pilar 1 — Onde Integrar IA (priorizado por ROI real)

TIER S — Fazer agora (1-2 semanas, impacto alto, esforço baixo)

2.1 IA Contextual com RAG Local — MAIOR ROI

O que é: Injetar contexto real do SQLite em todo prompt. Hoje: `SYSTEM_PROMPT + mensagens`. Depois: `SYSTEM_PROMPT + contextoDoUsuario + mensagens`.

Por que faz sentido: Todos os apps vencedores fazem (StudyEdge, Bevinzey, Plonor). Seu `getProfileStats()` e `carregarMaterias()` já existem — é só serializar. Sem isso, a IA nunca é "sua".

Como implementar (Groq + SQLite local, zero backend):

```
// src/services/aiService.js export async function enviarMensagemContextual(userId, mensagens)
{ const [stats, materias, settings] = await Promise.all([ getProfileStats(userId),
carregarMaterias(userId), getUserSettings(userId) ]); const fracas = materias.filter(m => {
const p = m.topicos.filter(t=>t.estudado).length / (m.topicos.length||1); return p < 0.5;
}).map(m=>m.nome).join(", ") || 'nenhuma'; const context = `CONTEXTO USUARIO:
${materias.length} materias, ${stats.topicosConcluidos}/${stats.totalTopicos} topicos,
${stats.totalHoras}h totais, streak ${stats.streak} dias, meta semanal
${settings.weeklyGoalMinutes}min (${stats.minutosSemana}min feitos), materias fracas:
${fracas}, melhor dia: ${stats.melhorDia}.`; const messages = [ {role:'system', content:
SYSTEM_PROMPT + '\n' + context}, ...mensagens.map(m=>({role:m.role, content:m.text})) ]; // ...
```

```
fetch Groq igual }
```

Custo: +300 tokens/request. Groq `llama-3.3-70b` aguenta. **Esforço:** 1 dia. **Risco:** zero.

2.2 Gerador de Tópicos/Flashcards por IA — Feature que mais retém

O que é: Botão "■ Gerar tópicos com IA" no `CreateMateriaModal.jsx`. Usuário digita "Biologia - Fotossíntese" → IA retorna JSON `[{nome: "Cloroplasto", estudado:false}, ...]` editável antes de salvar.

Precedente: StudyPLNR extrai capítulos de PDF/foto em segundos, SmartStudy gera flashcards de slides, Bevinzey Gera cards de qualquer documento. Dado duro: <16% dos usuários continuam após 1 semana quando criação é 100% manual (Mindomax 2025). Sua criação hoje é 100% manual.

Como implementar:

```
// aiService.js export async function gerarTopicos(nomeMateria) { const resp = await
fetch(BASE_URL, { body: JSON.stringify({ model: MODEL, messages: [ {role:'system', content:
'Gere 5-10 tópicos atômicos para a matéria. Retorne JSON array: [{\"nome\":\"...\"}], Tópicos:
curtos, especificos, sem repetição.'}, {role:'user', content: nomeMateria} ], response_format:
{type:'json_object'}, max_tokens: 400 }) }); // validar JSON, filtrar nome.trim(), limitar 10 }
```

Reusa `criarMateria(userId, nome, topicosGerados)`. **Esforço:** 1-2 dias (prompt + JSON parse + botão).

2.3 Quiz de Revisão Ativa dentro do DetailScreen

O que é: Após `stopStudy()` (que já existe), oferecer: "Gerar 5 perguntas sobre [tópicos estudados]?" Groq cria MCQs com 1 correta + 3 distratores plausíveis (como Memdora faz). Salvar erros.

Por que funciona: Recall ativo > releitura. Studentify/Plonor logam todo erro e repriorizam planner — é exatamente o que `toggleTopicoEstudado` deveria alimentar, mas hoje só flipa `bool`.

Como implementar: - Estender `topicos` para `{nome, estudado, dificuldade: 1-3, erros: number}` ou nova tabela `quiz_attempts(subject_id, topico_index, acertou, created_at)` - Groq prompt: "Crie 5 MCQs sobre X, 4 alternativas, indique correta, estilo ENEM/vestibular" - Log `erros` para alimentar FSRS/planner

TIER A — Fazer em 1 mês (médio esforço, diferencia o app)

2.4 FSRS-6 Spaced Repetition — Substitui `estudado: bool`

O que é: Trocar boolean por algoritmo de memória. FSRS-6 (lançado 2025, versão 6 com 21 parâmetros) é 99.5% mais preciso que SM-2 e exige 20-30% menos revisões para mesma retenção. Modela `difficulty`, `stability`, `retrievability` por card.

Precedente: Memdora, Bevinzey, SmartStudy migraram de SM-2 (1987, SuperMemo) para FSRS-6 em 2025. Benchmark com 10k coleções / 350M revisões comprova.

Como implementar no seu schema (offline, sem backend, client-side):

```
-- Opção A: estender JSON (compatível) topicos [TEXT] -- JSON: [{nome, fsrs:{difficulty,
stability, due_date, retrievability, reps}, estudado}] -- Opção B: nova tabela (mais queryável)
CREATE TABLE cards ( id INTEGER PRIMARY KEY, subject_id INTEGER NOT NULL, topico_index INTEGER
NOT NULL, due_date TEXT NOT NULL, stability REAL NOT NULL DEFAULT 0, difficulty REAL NOT NULL
DEFAULT 5, retrievability REAL NOT NULL DEFAULT 0.9, reps INTEGER NOT NULL DEFAULT 0, FOREIGN
```

```
KEY(subject_id) REFERENCES subjects(id) ON DELETE CASCADE );
```

Lib: `fsrs.js` (port JS, roda offline). Seu `ProgressScreen` já calcula `datasUnicas` e `streak` — metade do trabalho. Mostrar `vencem hoje: 12` no Home + notificação `expo-notifications`.

Regra de ouro (Sailer & Homner 2020): gamifique *consistência* (dias revisando), NUNCA *performance* (nota no quiz). Recompensar nota distorce o algoritmo.

2.5 Planner Adaptativo que se Reconstrói Sozinho

O que é: Hoje usuário cria matérias soltas sem data. Planner pergunta "Prova 15/09, 2h/dia" e gera blocos 30min (como Minerva propõe) distribuídos por dificuldade + proximidade da prova, e se reconstrói se faltar sessão.

Precedente: StudyPLNR rebuilds após cada sessão, StudyEdge replaneja se faltar 2 sessões (detecta gap e redistribui em 8 dias), Conch AI usa calendar + kanban. É a feature #1 de retenção em 2025.

Como implementar (SQLite local + Groq):

```
CREATE TABLE study_plan ( id INTEGER PRIMARY KEY AUTOINCREMENT, user_id INTEGER NOT NULL,
subject_id INTEGER NOT NULL, planned_date TEXT NOT NULL, duration_minutes INTEGER NOT NULL
DEFAULT 30, status TEXT NOT NULL DEFAULT 'pending', -- pending/done/missed/skipped FOREIGN
KEY(subject_id) REFERENCES subjects(id) ON DELETE CASCADE );
```

- `aiService.gerarPlano(userId, examDate, hoursPerDay)` → Groq distribui horas por dificuldade (`topicos.length` + erros)
- Ao `registrarSessao()`, rodar `checkAndReplan()` — se `missed>2`, Groq redistribui
- UI: expandir `HistoricScreen` + `ProgressScreen` com Calendar view (use `react-native-calendars`)

2.6 Tutor Socrático (hints-first), não resposta direta

O que é: Mudar `ChatScreen` de "responde tudo" para "dá dica antes da resposta, recompensa quem tenta".

Precedente: Studentify Homework Mode: "quer dica ou solução completa? Mostra raciocínio". Pesquisas: progressive disclosure (revelar resposta em partes) = +22% retenção vs full reveal. Dunlosky et al.: retrieval practice é 1 de 2 técnicas "high utility".

Como implementar: - Novo `SYSTEM_PROMPT_TUTOR` = "... Sempre ofereça 1 dica antes da resposta completa. Pergunte: Quer mais uma dica ou ver a solução? Recompense tentativa." - UI: botões `[■ Dica]` `[Ver resposta]` no bubble do bot (reaproveita `s.bolhaBot`) - Log `hint_used` para ajustar `difficulty` do tópico

TIER B — Só se sobrar tempo (alto esforço, legal mas não essencial agora)

2.7 Ingestão PDF/Foto → Matéria Automática

Usar `expo-image-picker` (já tem) + `expo-document-picker` + Groq Vision (`llama-3.2-vision`) ou OCR local (`expo-mlkit-ocr`). Usuário fotografa apostila → extrai tópicos. StudyPLNR/SmartStudy fazem, mas exige lidar com PDF parsing e custo vision. Deixe pra Fase 2.

O QUE NÃO FAZER — Encheção que parece inovação mas não paga

Ideia que vão te vender	Por que não fazer agora
Avatar 3D que cresce ■→■ (EchoStudy)	Caro de animar (Reanimated + 3D), meta-análise gamificação: $g=0.49$ só se for streak, avatar não melhora retenção
Chat de voz com IA	Consome bateria, quebra offline, seu público estuda em silêncio (87% cita notificação como distração, imagina voz)
Season-Based UI (troca visual por estação - Behance 2026)	Só ilustração, sem impacto em conclusão de tópico. É firula de portfólio
Previsão de nota com IA (Cortex "predicted exam score")	Chute estatístico que desmotiva se erra. Nenhum app acertou isso sem dados massivos
Gamificar XP por nota	Distorce FSRS. Pesquisa é clara: recompensa por <i>presença</i> gera engajamento superficial

3. Pilar 2 — Recursos Visuais que Funcionam (não só instagramável)

O que já funciona no Studify

Dark theme #090E1F / #111832 / #6F52FF / #8A68FF, LinearGradient, cards borderRadius 16, expo-blur/expo-linear-gradient. Está acima da média, **não precisa redesign total**.

5 upgrades com ROI comprovado

3.1 Progress Arc + Anel Diário (Home)

- **Referência:** Studia (Orbix Studio) fez 72% como arco curvo, não stat seco — sensação de momentum. Odyssey tem `DailyProgressCard` com ring. Focus Flow usa timer como elemento central.
- **Pro Studify:** Substitua `ProgressCards` (hoje texto puro) por `react-native-svg` arc que aponta pra `getWeeklyGoalProgress().percent`. Ex: anel 0-100% com `currentHours/goalHours` + animação `withTiming`. Usuário abre app e vê movimento, não número. Lib: `react-native-svg` (já deve vir com expo).

3.2 Heatmap 90 dias + Radar de Consistência

- **Referência:** Odyssey `StudyHeatmap.jsx` + `WeekdayConsistencyCard` (radar), LogIt analytics heatmap, Minerva checklist 30min blocks, GitHub-style.
- **Pro Studify:** Seu `ProgressScreen` já tem gráfico 7 dias barras. Estenda para heatmap 90 dias usando `carregarHistorico()` → mapear `started_at` -> `intensidade (minutos)`. Radar: `diaMap` que já calcula `melhorDia` → transformar em 7 eixos Dom-Sab. **Lib:** `react-native-gifted-charts` ou `victory-native` (recharts não existe nativo). Custo: 3 dias, impacto visual gigante.

3.3 Hierarquia de Syllabus (Subject → Topics com progresso)

- **Referência:** LogIt `Subject → Chapter → Topic → Subtopic` tree expansível com progresso por nível e barra.
- **Pro Studify:** Evoluir `CardMateria` para mostrar `5/8 tópicos (63%)` + barra fina `height:4` + cor semântica: verde `#10B981` fácil, amarelo `#F59E0B` médio, vermelho `#EF4444` atrasado (LogIt design system). Ao expandir, listar tópicos com checkbox + `due_date` FSRS (vermelho = vence amanhã). Seu `calcGlobalStats` já calcula.

3.4 Micro-interações com Reanimated 3 (não só opacity)

- **Referência:** LogIt usa `Reanimated 3` 60fps + spring + haptics. Seu `BottomNav/CardMateria` usam só `activeOpacity 0.7`.
- **Pro Studify:** Adicionar `react-native-reanimated` + `react-native-gesture-handler` (já tem) + `expo-haptics` para: check tópico `scale 0.8→1.2` + haptic light, progress bar `withTiming(percent, {duration:600})`, flip card 3D. Esforço baixo, percepção "app premium" sobe 200%. Khan Academy 2025 fez e +38% lesson completion.

3.5 Dashboard "Smart Next Steps" (Khan Academy pattern)

- **Referência:** Khan Academy 2025 redesenhou dashboard em torno de `Smart Next Steps` (IA recomenda próxima atividade) + `Energy Points` + `Mastery Levels (Familiar→Proficient→Mastered)` → +69% skills mastered/mês.
- **Pro Studify:** No `HomeHeader` abaixo da busca, card "Próximo passo inteligente":
- `streak==0` → "■ Volte hoje e mantenha seu streak!"
- `tem_topico_com_due_date==hoje` → "■ Revisar: Mitose (vence hoje)"
- `materia 0%` → "■ Comece: História" Usa mesma lógica do planner. Não é mais UI, é orquestração de dados que já tem. Espaço: 1 card, 1 linha.

Paleta fixa (não use roxo pra tudo):

```
Primary #6F52FF | Secondary #8A68FF | BG #090E1F / #111832 Sucesso #10B981 | Alerta #F59E0B | Erro #EF4444 | Texto #F4F6FF / #7F8AB7
```

4. Pilar 3 — Métodos de Módulos / Arquitetura (como não virar spaghetti)

Estado Atual — Onde dói

- `App.js` com `Stack.Navigator` manual (9 screens), sem Expo Router
- `subjectsDb.js` 900 linhas fazendo 5 domínios (subjects, sessions, chat, badges, settings)
- `authDb.js` + `subjectsDb.js` abrem o MESMO `studify.db` com `getDb()` duplicado (race condition real)
- `HomeScreen.js` 300 linhas com 6 `useState` de modais + helpers espalhados em `components/home/helpers.js`
- `src/styles/*` separado de componentes (dificulta mover feature)

Case real: App com 5→38 telas em 14 meses colapsou com estrutura type-based (um `components/` gigante, API em screens, `App.js` com string navigation). É exatamente o caminho que Studify está trilhando.

Modelo Recomendado — ExFeAr (Expo Feature-Sliced) + FSD

Regra de ouro: Routes → Features → Shared/Core. Feature nunca importa outra Feature. Shared nunca importa Feature. Um sentido só.

Validado em Expo 54 (2025) e usado em produção. É o que `expo-nativewind-sample` (Clean Architecture) e `ExFeAr-Guideline v2.0` documentam.

Estrutura Alvo (migração gradual, não big bang)

```
Studify/ ■■■ app/ # SÓ ROTAS (Expo Router - futuro) ■■■ _layout.tsx # NavigationContainer +
ErrorBoundary (hoje em App.js) ■■■ (auth)/ ■■■ _layout.tsx ■■■ login.tsx # thin
shell -> features/auth/LoginFeature ■■■ register.tsx ■■■ (app)/ ■■■ _layout.tsx #
Auth guard (hoje bootstrapSession em App.js) ■■■ (tabs)/ ■■■ _layout.tsx # BottomNav
■■■ index.tsx # Home ■■■ progress.tsx ■■■ profile.tsx ■■■ chat.tsx ■■■
detail/[id].tsx # Detail ■■■ src/ ■■■ features/ # CADA DOMÍNIO É UM MÓDULO FECHADO ■■■
study/ # Home + Detail + timer + sessions ■■■ components/CardMateria, MateriaSections,
ProgressCards ■■■ hooks/useStudySession.ts # extrai interval de DetailScreen ■■■
hooks/useMaterias.ts # extrai loadHomeData ■■■ services/studyService.ts #
registrarSessao, toggleTopico ■■■ types.ts ■■■ planner/ # futuro - study_plan + IA
■■■ chat/ # ChatScreen + aiService contextual ■■■ progress/ # ProgressScreen + stats
+ heatmap ■■■ components/Heatmap, weeklyChart ■■■ hooks/useProgressStats ■■■
■■■ auth/ # Login/Register + authDb ■■■ entities/ # Modelos puros (sem logica de DB) ■■■
■■■ subject.ts # type Subject {id, nome, topicos: Topic[], fixada} ■■■ session.ts ■■■
■■■ badge.ts ■■■ shared/ # SÓ O QUE 2+ FEATURES USAM ■■■ components/Icon, CustomInput,
CustomButton, BottomNav, ErrorBoundary ■■■ hooks/useUserId.ts ■■■
utils/formatTime.ts # extrair de DetailScreen ■■■ core/ # Infra - ninguém de negócio importa
■■■ db/ ■■■ client.ts # getDb() ÚNICO (hoje duplicado!) ■■■ migrations.ts # toda
execAsync + seed badges ■■■ api/ ■■■ groqClient.ts # fetch Groq centralizado (hoje se
em aiService) ■■■ queryClient.ts # TanStack Query client ■■■ config/env.ts #
EXPO_PUBLIC_GROQ_API_KEY tipado
```

3 Regras que Evitam Dor (FSD + ESLint)

1. **Quebre `subjectsDb.js` AGORA:** Crie `core/db/client.ts` único. Hoje `authDb` e `subjectsDb` chamam `SQLite.openDatabaseAsync('studify.db')` separados — se um faz `PRAGMA journal_mode = WAL` e outro cria tabela ao mesmo tempo, dá race. Um `client.ts` com singleton + `initialized` flag resolve. Seu código já tem `dbPromise = null` auto-recuperação — centralize lá.

2. **`app/` importa de `features/`, nunca contrário:** Seu `DetailScreen` hoje `import {getMateriaById} from '../services/subjectsDb'`. Futuro: `app/detail/[id].tsx` (10 linhas, só pega `id` da rota) renderiza `features/study/DetailFeature` que importa `core/db`. Screen não tem lógica. É o que permite `Expo Router typedRoutes: true`.

3. **Husky + ESLint boundary (obrigatório quando passar de 2 devs):**

```
// eslint.config.js (flat) { files: ['src/features/**/*.{ts,tsx}'], rules: {
  'no-restricted-imports': ['error', { patterns: [{group: ['@/features/*'], message: 'Feature não
importa outra Feature. Use shared/ ou core/'}] } ] }
```

`git commit` bloqueia se `features/study` importar `features/chat`. Sem isso, consertar `Profile` quebra `Home` (circular hell documentado).

Tipo de estado	Onde está hoje	Use
Server/local DB	<code>useState</code> + <code>navigation.addListener('focus', loadHomeData)</code> manual	<code>TanStack Query</code> — <code>useQuery(['materias', userId], carregarMaterias)</code> já faz cache, refetch, loading/error. Substitui 60% do Redux
Client UI	<code>useState</code> pra <code>modalVisible</code> , <code>busca</code> , <code>editNome</code>	<code>useState</code> local ou <code>zustand</code> (1KB, sem Provider) pra <code>BottomNav</code> tab ativa
Persistido	<code>AsyncStorage</code> sessão + SQLite estruturado	<code>expo-sqlite</code> pra queries/indexes, <code>MMKV</code> pra prefs rápidas (sync, mais rápido que AsyncStorage)

Regra: Se TanStack Query resolve, use Query. Se é UI local, `useState`. Só use Redux/Zustand global pra coisas tipo `theme`, `user`.

Config Expo Moderna (app.config.ts vs app.json)

Hoje: `app.json` estático. **Migração:** `app.config.ts` (TS) permite `process.env` por ambiente (dev/staging/prod), `newArchEnabled: true` (Fabric/TurboModules), `reactCompiler: true` (elimina `useMemo/useCallback` manual), `typedRoutes: true`.

```
// app.config.ts export default ({config}) => ({ ...config, name: 'Studify', slug: 'studify', newArchEnabled: true, experiments: {typedRoutes: true, reactCompiler: true} })
```

Migração em 3 Passos (sem parar feature)

Semana 1 — Zero breaking: - [] Extrair `core/db/client.ts` (mover `getDb()` de `authDb`+`subjectsDb` pra um lugar só) - [] Criar `shared/utils/formatTime.ts` (mover `formatarTempo`, `formatarCronometro`, `calcGlobalStats` de `Detail/Home`) - [] Criar `core/api/groqClient.ts` (centralizar `fetch` + `BASE_URL` + `API_KEY` check)

Semana 2 — Isolar Study: - [] Criar `features/study/hooks/useMaterias.ts` (extraí `loadHomeData` + `carregarMaterias` + `carregarHistorico`) - [] Criar `features/study/hooks/useStudySession.ts` (extraí `studying/paused/elapsed/intervalRef` do `Detail`) - [] Mover `CardMateria`, `MateriaSections` pra `features/study/components/`, `HomeScreen.js` vira shell magro

Quando bater 15 telas ou 2 devs: - [] `npx expo install expo-router` + migrar `Stack` → `app/` file-based routing - [] `babel module-resolver` alias `@features`, `@shared`, `@core` - [] Husky pre-commit `npm run lint && npm run type-check`

5. Roadmap Enxuto — O que fazer na ordem

FASE 1 — 2 semanas (quick wins que usuário sente na 1ª abertura)

- [] **IA contextual** (injetar stats no prompt) — 1 dia — *impacto: IA finalmente útil*
- [] **Botão "Gerar tópicos com IA"** no CreateMateriaModal — 2 dias — *impacto: -80% fricção de criação*
- [] **Anel de progresso** (`getWeeklyGoalProgress`) no Home — 2 dias — *impacto: usuário vê momentum*
- [] **Quebrar `subjectsDb.js` → `core/db/client.ts`** — 1 dia — *impacto: elimina race, base pra escalar*
- [] **Heatmap 7→30 dias** no ProgressScreen — 3 dias — *impacto: prova visual de consistência*

Só Fase 1 já deixa Studify à frente de 80% dos study apps (que são CRUD + chat genérico).

FASE 2 — 1 mês (vira app inteligente)

- [] **FSRS-6** + `due_date` por tópico + notificação `expo-notifications` "12 cards vencem hoje"
- [] **Quiz MCQs pós-sessão** + log `quiz_attempts` + repriorização
- [] **Planner adaptativo** (`study_plan` table + `checkAndReplan()` com Groq)
- [] **Migrar `features/study` modular** + `TanStack Query` (substitui `addListener('focus')`)

FASE 3 — 2 meses (diferencial de mercado, só se validar retenção)

- [] **PDF/Foto → tópicos** (`expo-document-picker` + `expo-image-picker` + Groq Vision/OCR)
- [] **Tutor hints-first** + progressive disclosure (revelar resposta em partes)
- [] **Expo Router + typedRoutes + Husky boundaries** + `react-native-reanimated` micro-interações

6. Referências Pesquisadas (2025-2026, produção real)

IA & Aprendizado: - Bevinzey — SM-2 spaced repetition + 16 módulos IA, retrieval practice interleaving (g=0.42) - StudyPLNR — Adaptive daily plan + PDF/photo import + Calendar sync - Studentify — 6 modos de tutor, hints-first, spaced repetition 85-90% retention target - StudyEdge AI — Week-by-week plan + minute-by-minute blueprint + auto re-planning - Cortex Surgery AI Planner — SM-2, Plan Builder drag reorder, 90-day heatmap, AI Coach weekly review - Minerva (SBIE 2025, 50 estudantes, UTAUT) — learning cycles adaptativos + habit tracker + dashboard por matéria - Mindomax (05/2025) — FSRS-6 99.5% mais preciso que SM-2, 20-30% menos revisões, meta 85-90% retention - Memdora (arXiv 2025) — 17 interaction types, FSRS-6 client-side, single-gesture capture, effort-based rewards

Visual & UX: - Odyssey v2.0 (React 19 + Recharts + Supabase) — Dashboard daily/weekly, progress ring, weekday radar, heatmap, rewards - LogIt (React Native + Reanimated 3) — Hierarchical syllabus Subject→Chapter→Topic, flipped clock, 3D flashcards - Studia (Orbix Studio, Muzli 2026) — Progress arc 72% como momentum, glassmorphism, peach gradients - Khan Academy AI-Native Dashboard (2025) — Smart Next Steps + Energy Points + Mastery Levels → +38% completion, +69% skills/mês - Focus Flow (Medium 2025) — Timer central + ambient mixer + minimal tasks → +153% focus time, +533% daily return

Arquitetura Modular: - FSD (Feature-Sliced Design) — layers `app/features/entities/shared`, ESLint boundaries, `no-restricted-imports` - ExFeAr v2.0 (Toni Maxx, Expo 54 + Expo Router) — "Routes map to features. Features use

shared. Shared never depends on features." + `features/shared` para cross-feature domain - expo-nativewind-sample (Clean Architecture) — `modules/*/ {components, hooks, repositories, useCases, store}` + WatermelonDB + Redux Toolkit - Hashnode: Scalable RN Architecture FSD & Expo (04/2026) — `app.config.ts` dynamic, React Compiler, Husky pre-commit - Medium: Modular Expo Folder Structure 2025 — `modules/auth/{components, hooks, services}` + Zustand + Axios + Maestro

7. Como usar este doc com seu amigo

1. **Marquem Fase 1 juntos** — são 5 itens, todos cabem em 2 semanas. Decidam quem pega IA (api) e quem pega visual (ring/heatmap).
2. **Estimem FSRS vs Planner** — FSRS é mais técnico (muda schema), Planner é mais produto (muda UX). Qual dor do usuário de vocês é maior: "esqueço o que estudei" (FSRS) ou "não sei o que estudar hoje" (Planner)?
3. **Arquitetura:** combinem de fazer `core/db/client.ts` ANTES de qualquer feature nova — é 1 dia e evita dor futura. O resto pode ser gradual.

Pergunta de ouro pra discussão: *"Se o usuário só pudesse ter UMA dessas features novas, qual faria ele pagar pelo app?"* — Geralmente é **Planner adaptativo** ou **FSRS**. Comecem por ela.

Gerado a partir de análise do codebase `Studify/` (`App.js`, `aiService`, `subjectsDb`, `authDb`, `Home/Detail/Chat/Progress`) + pesquisa web 2025-2026 (8 queries, 40+ fontes). Sem mocks, tudo referenciado em produção.